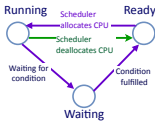


CONCEPTS OF CONCURRENCY

Parallelism: Subprograms run simultaneously, goal: faster programs
Concurrency: interleaved execution of programs, goal: simpler programs
Process: Program under Execution, Heavyweight, own address space, OS needs process control
Thread: **Parallel** sequence in process, Lightweight, same address space, separate stack and registers

THREAD SCHEDULING

core sharing: Run more threads than cores, On waiting: Release core for other ready threads
scheduler: "threads are on mercy of scheduler"
Context switch: **Synchronous** Thread waiting for condition queues itself and gives core free
Asynchronous context switch due to an external event (hardware or timer interrupt), prevents thread from permanently occupying.
Cooperative Multi-Tasking (scheduling): Threads must explicitly initiate context switches, scheduler can't interrupt.
Preemptive Multi-Tasking: scheduler can asynchronously interrupt thread via timer interrupt
Time-Sliced Scheduling (preemptive): Each thread has the processor for maximum time interval



JVM THREAD MODEL

Java: single process system
Java Virtual Machine (JVM): is a process
threads: main thread which calls the main function, programmer can start more threads, Subsystems / Runtime System also start their own threads (e.g. RMI, AWT, Garbage Collector)
Termination: after all (not daemon) threads are finished or System.exit/Runtime.exit
Thread class: override the run method of the Runnable interface with the code to be executed in parallel.

```
create a start a thread // Multi-Thread Examples Java
// explicit Runnable implementation // no synchronization
class myRunnable implements Runnable { public static void main(String[] args) {
    @Override var a = new Thread(() -> multiP("A"));
    public void run() { var b = new Thread(() -> multiP("B"));
        /* thread behavior */ a.start(); b.start();
    } } System.out.println("main finished");
var myT = new Thread(new myRunnable()); } // varying result, non-deterministic
// named function scheduler
var myT = new Thread(() -> // with synchronization (a and b from
namedFunction()); above);
// anonymous function (lambda) System.out.println("Threads start");
var myT = new Thread(() -> { a.start(); b.start(); // ...
    /* thread behaviour */ a.join(); b.join();
}); // CF Lambda lambda with => arrow System.out.println("Threads joined");

// start the thread (not run!)
myT.start();
```

Thread Java & .NET
t.start() == **t.start()** :
t.join() == **t.join()** : wait till thread finished, timeout: join(ms),
t.sleep(ms) == **t.sleep(ms)** : join+sleep -> Timed-Waiting state,
t.wait() : gives processor+lock free,
t.yield() : scheduler can release processor, to Ready state, does not release lock
t.setDaemon(true) == **isBackground** == **true** :
t.interrupt(): for cooperative canceling. Use when Cancel-Policy of threads is known (know what thread does after it was interrupted)
InterruptedException: thrown on interrupt() while sleep/wait/join.
t.interrupted() **t.isInterrupted()**: check interrupted flag
currentThread(): returns Thread, reference to current executing thread instance

Java Thread States:
Blocked (blocked and waiting for a monitor lock) , **New** (not yet started) , **Runnable** (Ready to run or running) , **Terminated**, **Timed_Waiting** (waiting with a specified waiting time **Thread.sleep(ms)/Thread.join(ms)**) , **Waiting**

MONITOR SYNCHRONIZATION

Non-determinism: threads run interleaved or parallel
Synchronization (restriction of concurrency): for deterministic behavior.
Communication between threads: Sharing access to fields and the objects they refer to. Efficient, but problems: **Thread interference** and **memory consistency errors**.
Critical Section: code must be executed by only 1 thread at a time. Implementation with **mutual exclusion**.

happens-before relationship: result is same as if executed synchronously (visibility + threadsafe)
Java synchronized
synchronized: Body of method with the **synchronized** keyword is a critical section. Guarantees memory consistency and a **happens-before relationship** . Other threads are **blocked** until the current thread is done with the object. Every object has a **Lock (Monitor-Lock)** . Maximum 1 thread can acquire the lock. Entry of a **synchronized** method acquires the lock of the object, the exit releases it.
public synchronized void deposit(int amount) { this.balance += amount; }
synchronized can also be used within a method, the **object that should be locked** must be specified.
synchronized(this) { this.balance += amount; } (unhandled exceptions exit sync. block)

Monitor Lock
Monitor for **internal mutual exclusion**. Only one thread operates at a time in Monitor, has **fairness-problems** and **no shared locks**.

Recursive Lock: thread can acquire same lock through recursive calls, free by last release
Busy Wait: is repeatedly checking a condition without giving CPU free **while** (Locked) { } (does not work)
Wakeup signal: Signalling a thread in Monitor (inner waiting room). **notify()** and **notifyAll**
notify(): signals any waiting thread. Turnstile
One-in/One-Out: change applies to only one thread, Only a single waiting thread can continue
Uniform Waiters: Only 1 semantic condition for every thread, condition interests every waiting Thread
notifyAll(): wakes up all threads (from inner waiting room to outer waiting room to wait for entry in monitor)
signal and continue: signaling (notifying) thread releases only after leaving synchronized, awakened thread comes to outer waiting room.
IllegalMonitorStateException: thrown if notify/notifyAll/wait used outside synchronized.

```
// Java // CF .NET
class BankAccount { class BankAccount {
    private int balance = 0; private decimal balance;
    public synchronized void withdraw private object syncObject = new();
        (int amount) public void Withdraw(decimal amount) {
            throws InterruptedException { lock (syncObject) {
                while (amount > balance) { while (amount > balance) {
                    wait(); // wait on a condition Monitor.Wait(syncObject);
                } }
                balance -= amount; balance -= amount;
            } }
        } // release / leave monitor }
    public synchronized void deposit public void Deposit(decimal amount) {
        (int amount) { lock (syncObject) {
            balance += amount; balance += amount;
            notifyAll(); Monitor.PulseAll(syncObject);
        } }
    } }
```

HAZARDS AND CONDITIONS OF CORRECTNESS
SPECIFIC SYNCHRONIZATION PRIMITIVES
THREAD POOLS
Wovon hängt die Anzahl Tasks bei beiden Verfahren ab? o .NET Parallel For: Anzahl freier Worker Threads o Java ForkJoinPool: Array-Länge und Threshold
PARALLELFOR (.NET)
FORKJOINPOOL (JAVA)
ASYNCHRONOUS PROGRAMMING
MEMORY MODELS